

AI Weekly Schedule Optimizer

Planning a week as search + constraint satisfaction

CSC 480 Final Project

The Problem

- People juggle **fixed commitments** (work, classes) and **flexible tasks** (study, chores, projects)
- Flexible tasks have a **duration, deadline, priority, and preferences**
- Planning by hand is hard: many tasks, competing deadlines, limited time
- A normal calendar app *stores* events — it doesn't *decide where things go*

The AI Connection

We model the week as a **constraint-satisfaction problem (CSP)**:

- The week is a grid of discrete **time slots**
 - Each flexible task is a **variable** — its values are possible placements
 - We **search** for an assignment that satisfies hard constraints...
 - ...and **optimizes** soft preferences
- “ The system searches over many possible assignments, detects conflicts,
backtracks, and compares schedules with a scoring function. ”

How Time Is Modeled

- The week → **10-minute slots**, Mon–Sun, 8 AM to midnight
- **Fixed events** permanently block their slots
- **Flexible tasks** must claim enough free slots to cover their duration

Why 10 minutes? Fine enough to be realistic, coarse enough to keep the search space small → "*plan my week*" becomes a **finite search problem**.

The Rules: Hard Constraints

A schedule is **only valid** if every task:

- does **not overlap** a fixed event or another flexible task
- finishes **before its deadline**
- stays **within available hours** (8 AM – 10 PM)

Both schedulers call the **same** `is_slot_valid()` check — identical rulebook.

What Makes a Schedule *Good*: Soft Constraints

We **score** valid schedules on preferences (points are tunable):

Reward	pts	Penalty	pts
Scheduled ($\times$ priority)	+100	Left unscheduled	-100
On a preferred day	+20	Late-night slot	-5
In a preferred time window	+20		
High priority placed early	+10		

Approach 1: Greedy Baseline

- Sort tasks by **priority, then deadline**
- Drop each into the **earliest open valid slots**
- Very fast and simple but short sighted

Issue: Never reconsiders, ends up having all tasks in one day when the tasks could be done in multiple days instead

Approach 2: Backtracking CSP Scheduler

The "AI" centerpiece. Each task is a variable; its domain is every legal **contiguous block**, plus a **"skip"** option.

Real CSP techniques:

- **Most-constrained-first** variable ordering
- **Best-score-first** value ordering
- **Branch-and-bound** pruning

Because it can **undo** bad early choices, it finds schedules greedy misses.

Explaining Failures

When a task can't be scheduled, we say **why** — in plain English:

- **Intrinsically impossible**

"needs 20h in one day, but only 14h is available per day"

- **Crowded out**

"no free 2h block remained before its Mon 10 PM deadline — other tasks took the time first"

Evaluation: How We Tested

- **Three weeks** of increasing difficulty: easy, medium, **hard**
- The **hard** week is engineered so a full valid schedule *exists*, but greedy boxes itself in
- An **independent validator** re-audits every output for rule violations
 - we don't just trust the schedulers

Live Demo

```
python evaluation.py    # run both schedulers on all three test weeks
```

```
HARD WEEK    (a full valid schedule exists)
```

Scheduler	Scheduled	Violations	Score	Time(ms)
Greedy	2/3	0	450	0.57
CSP	3/3	0	660	6.64

```
    Greedy dropped Tight Errand: no free 2h block remained  
    before its Mon 10:00 deadline -- other tasks took the time first
```

Results

Week	Greedy	CSP	Outcome
Easy	2/2 · 330	2/2 · 342	CSP wins on preferences
Medium	3/3 · 680	3/3 · 692	CSP wins on preferences
Hard	2/3 · 450	3/3 · 660	Greedy fails; CSP fits all (+210)

- **0** hard-constraint violations anywhere
- Runtimes in **single-digit milliseconds**

The Hard Week, Step by Step

- "Tight Errand" can **only** go Mon 8–10 (deadline Mon 10 AM)
- Greedy schedules higher-priority tasks first → grabs Mon 8–10
- → "Tight Errand" has **nowhere to go**
- CSP recognizes the conflict, moves the big tasks, **reserves Mon 8–10**

Architecture

Module	Responsibility
<code>models</code> · <code>slots</code>	Data types & the time grid
<code>constraints</code>	Shared hard-constraint rules
<code>greedy</code>	Baseline scheduler
<code>backtracking</code>	CSP scheduler
<code>score</code>	Soft-constraint scoring
<code>explain</code>	Why a task was dropped
<code>evaluation</code>	Benchmark suite + validator

Design Choices

- **Contiguous block per day** — a task runs in one sitting (e.g. a 3-hour study block), which is more realistic than scattering 10-minute fragments
- **Hand-tuned weights** — transparent and explainable; every point is traceable
- **Skip option** — when not everything fits, it degrades gracefully and *reports* what it dropped instead of failing

Each is a deliberate trade-off — and a clean hook for future work.

Takeaways

Search + CSP + heuristics turn a messy planning chore into a **solvable, optimizable** problem.

We didn't just *claim* the AI approach is better — we **measured it**:

- Schedules **more tasks** (3/3 vs 2/3 on the hard week)
- Respects **more preferences** (higher score every week)
- **Zero** rule violations, verified independently

Thank you.